Chopsticks Framework

Performance Optimization Report

Version 2.1.0 **Zero-Allocation Release**

100%	27%	0 B	4x
Less Sync Alloc	Faster Dispatch	Sync Path Alloc	Pools Added

Prepared by: Sean & Performance Team
Date: May 19, 2026
Classification: Internal Technical Documentation

Tests:	**163 passing**
Breaking:	**0 changes**
New Files:	**1 added**

Table of Contents

1 Executive Summary

To: Engineering Leadership & Client Development Teams
From: Sean & Performance Team
Re: Chopsticks Framework v2.1.0 — Zero-Allocation Release
Date: May 19, 2026

We are pleased to announce **Chopsticks Framework v2.1.0**, the **Zero-Allocation Release**. This version delivers on the promise of allocation-free synchronous message handling, eliminating 100% of heap allocations in the sync hot path.

Key Achievements:

- **100% allocation reduction** for synchronous handlers (312 B → 0 B)
- **27% faster** message dispatch (52 ns → 38 ns)
- **Full pooling** for all 4 promise source types
- **Pre-warming API** for optimal startup performance
- **Zero breaking changes** — fully backward compatible with V2.0.0

This release is **fully backward compatible** with V2.0.0 and V1.0. No code changes are required to upgrade. All 163 unit tests pass, and benchmarks confirm improvements across all measured scenarios.

For game developers running at 60 FPS: messaging now contributes **zero** to garbage collection pressure.

1.1 What's New in V2.1.0

1.1.1 Zero-Allocation Sync Path NEW

Synchronous handlers now bypass promise sources entirely:

- `TryHandle()` allocates **0 bytes**
- `Handle()` allocates **0 bytes**
- `TryHandleAsync()` allocates **0 bytes**

1.1.2 Full Promise Source Pooling NEW

All 4 non-sequential promise source types now pooled:

- `TryHandlePromiseSource`
- `TryHandleAsyncPromiseSource`
- `HandlePromiseSource`

- `HandleAsync()` allocates **0 bytes**

Direct result constructors store the `HandlingResult` inline instead of wrapping in a promise source.

1.1.3 Pre-Warming API NEW

New `PromiseSourcePools.PreWarm()` method:

```
// Call at startup
PromiseSourcePools.PreWarm();
```

Eliminates first-use allocation spike by pre-populating pools.

- `HandleAsyncPromiseSource`

Async handlers benefit from reduced allocation when pools are warm.

1.1.4 Backward Compatible

- No breaking changes from V2.0.0
- No code modifications required
- All existing tests pass
- Drop-in replacement upgrade

1.2 Version Comparison at a Glance

Sync TryHandle()

V1.0	V2.0	V2.1
312 B	272 B	0 B

100% Reduction

Sync Handle()

V1.0	V2.0	V2.1
560 B	480 B	0 B

100% Reduction

Dispatch Latency

V1.0	V2.0	V2.1
52 ns	44 ns	38 ns

27% Faster

Multicast (5 handlers)

V1.0	V2.0	V2.1
344 B	272 B	272 B

21% Reduction

1.3 Memory Pressure Comparison

Scenario: 60 FPS Game Loop, 10 Message Types, Sync Handlers

V1.0.0	V2.0.0	V2.1.0
187 KB/sec	163 KB/sec	0 KB/sec
11.2 MB/min	9.8 MB/min	0 MB/min
GC every 5 sec	GC every 6 sec	NO GC from messaging

1.4 Breaking Changes Summary

> **✓ No Breaking Changes**
>
> **V2.1.0 is fully backward compatible with V2.0.0 and V1.0.**
>
> - No interface changes
> - No method signature changes
> - No behavior changes
> - Drop-in replacement upgrade
>
> **Optional improvement:** Call `PromiseSourcePools.PreWarm()` at startup for optimal performance.

2 Performance Benchmarks

All benchmarks were run using **BenchmarkDotNet v0.14.0** on:

OS: Windows 11 (10.0.26200)
Runtime: .NET 9.0.16 (9.0.1626.22923), X64 RyuJIT AVX2
GC: Concurrent Workstation
Iterations: 10 per benchmark (3 warmup)

2.1 Complete Version Comparison

Benchmark	V1.0.0	V2.0.0	V2.1.0	Total Δ
Sync TryHandle()	52 ns, 312 B	44 ns, 272 B	**38 ns, 0 B**	**27% / 100%**
Sync Handle()	65 ns, 560 B	55 ns, 480 B	**40 ns, 0 B**	**38% / 100%**
Sync TryHandleAsync()	55 ns, 312 B	46 ns, 272 B	**39 ns, 0 B**	**29% / 100%**
Sync HandleAsync()	70 ns, 560 B	60 ns, 480 B	**45 ns, 0 B**	**36% / 100%**
Multicast (5 handlers)	92 ns, 344 B	73 ns, 272 B	**62 ns, 272 B**	**33% / 21%**
Exception (single)	15 ns, 80 B	5 ns, 56 B	5 ns, 56 B	**67% / 30%**
Exception (array)	19 ns, 80 B	11 ns, 32 B	11 ns, 32 B	**42% / 60%**

> **Key Insight**
>
> **Sync handlers now allocate 0 bytes.** The direct result path bypasses promise source allocation entirely, storing the `HandlingResult` inline in the struct.

2.2 Promise Source Pooling Performance

Promise Source	New (no pool)	Pooled	Δ Speed	Δ Alloc
TryHandlePromiseSource (1K ops)	15 µs, 128 KB	12 µs, 0 B	+20%	−100%
TryHandleAsyncPromiseSource	18 µs, 128 KB	14 µs, 0 B	+22%	−100%
HandlePromiseSource	16 µs, 128 KB	13 µs, 0 B	+19%	−100%
HandleAsyncPromiseSource	17 µs, 128 KB	14 µs, 0 B	+18%	−100%
Real-world Chain	35 µs, 256 KB	28 µs, 0 B	+20%	−100%

3 V2.1.0 New Optimizations

This section details the three new optimizations introduced in V2.1.0.

3.1 Zero-Allocation Sync Path

Priority: P0 — Critical **Impact:** Eliminates ALL sync allocations

3.1.1 Problem

Even with V2.0.0 pooling, sync handlers still allocated promise sources:

```
// V2.0.0 — Still allocates (even if pooled)
HandlingResultPromise ISyncMessageHandler<TMessage>.TryHandle(TMessage message)
{
    var source = TryHandlePromiseSource.Rent();  // Pool hit or allocation
    source.Init(EvaluateHandle(message));
    return new HandlingResultPromise(source);    // Stores interface reference
}
```

3.1.2 Solution

Added **direct result constructors** that bypass promise sources entirely:

```
// V2.1.0 — Zero allocation
public readonly struct HandlingResultPromise
{
    private readonly IHandlingPromiseSource? _source;
    private readonly HandlingResult _directResult;
    private readonly bool _hasDirectResult;

    // NEW: Direct result constructor — zero allocation
    public HandlingResultPromise(HandlingResult result)
    {
        _directResult = result;
        _hasDirectResult = true;
        _source = null;  // No source needed!
    }
}

// Sync handler now uses direct result
HandlingResultPromise ISyncMessageHandler<TMessage>.TryHandle(TMessage message)
{
    return new HandlingResultPromise(EvaluateHandle(message));  // ZERO allocation!
}
```

✓ Client Impact

None. The API is unchanged. All existing code works without modification.

3.2 Promise Source Pooling (All Types)

Priority: P1 — Major **Impact:** Reduces async handler allocations

3.2.1 Problem

V2.0.0 only pooled `SequentialHandlingPromiseSource`. Four other sources were still allocated fresh:

- `TryHandlePromiseSource` — 128 B per use
- `TryHandleAsyncPromiseSource` — 128 B per use
- `HandlePromiseSource` — 128 B per use
- `HandleAsyncPromiseSource` — 128 B per use

3.2.2 Solution

Added `ConcurrentBag` pooling to all four types:

```
public class TryHandlePromiseSource : BaseHandlingPromiseSource<HandlingResult>
{
    private static readonly ConcurrentBag<TryHandlePromiseSource> Pool = new();
    private const int MaxPoolSize = 64;

    public static TryHandlePromiseSource Rent()
    {
        if (Pool.TryTake(out var source))
            return source;
        return new TryHandlePromiseSource();
    }

    public override void Dispose()
    {
        base.Dispose();
        if (Pool.Count < MaxPoolSize)
            Pool.Add(this);
    }
}
```

✓ **Client Impact**

None. Pooling is internal and transparent.

3.3 Pool Pre-Warming API

Priority: P2 — Minor **Impact:** Eliminates startup allocation spike

3.3.1 Problem

With pooling, the first N dispatches still allocate until the pool fills.

3.3.2 Solution

Created `PromiseSourcePools` utility class:

```
public static class PromiseSourcePools
{
    public static void PreWarm(int countPerPool = 16)
    {
        // Pre-creates and disposes instances to populate pools
        PreWarmTryHandlePromiseSources(countPerPool);
        PreWarmTryHandleAsyncPromiseSources(countPerPool);
        PreWarmHandlePromiseSources(countPerPool);
        PreWarmHandleAsyncPromiseSources(countPerPool);
    }
}
```

3.3.3 Usage

```
// Program.cs or Startup.cs
public static void Main(string[] args)
{
    PromiseSourcePools.PreWarm();  // Default: 16 per pool
    // or
    PromiseSourcePools.PreWarm(32);  // Custom count

    // ... rest of startup
}
```

> **Memory Impact**
>
> Pre-warming allocates upfront: 16 × 4 pools × 128 B = **8 KB** one-time cost.

4 V2.0.0 Optimizations (Reference)

For completeness, here is a summary of optimizations from V2.0.0 that remain in V2.1.0.

Ticket	Description	Impact
CHOP-001	SequentialHandlingPromiseSource pooling	−40 B per multicast dispatch
CHOP-003	Thread-safe SingletonResolution	Fixes race condition
CHOP-004	Thread-safe ContainedResolution	Fixes race condition
CHOP-005	Cached handler array	Eliminates array copy per dispatch
CHOP-006	ImmutablePromiseSource for statics	Fixes shared mutable state
CHOP-008	Zero-allocation exception enumeration	−24 to −48 B per exception access
CHOP-010	Single dictionary lookup	9% faster deregistration
CHOP-011	Binary search registration	O(log N) vs O(N log N)
CHOP-012	Linear exception accumulation	O(N) vs $O(N^2)$ allocation
CHOP-014	IDisposable on IHandlingPromiseSource	Enables pooling
CHOP-015	readonly struct HandlingResultPromise	Prevents defensive copies
CHOP-017	static readonly fields	Eliminates getter overhead

5 Migration Guide

5.1 Upgrading from V2.0.0 to V2.1.0

> **✓ No Changes Required**
>
> V2.1.0 is a **drop-in replacement** for V2.0.0.
>
> Simply update the package reference and rebuild.

5.1.1 Optional Improvements

Add Pre-Warming (Recommended)

```
// In Program.cs or Startup.cs
PromiseSourcePools.PreWarm();
```

Remove Custom Pooling

If you implemented your own pooling, you can remove it — it's now built-in.

5.2 Upgrading from V1.0.0 to V2.1.0

5.2.1 Required Changes

> **⚠️ Interface Change (from V2.0.0)**
>
> `IHandlingPromiseSource` now extends `IDisposable`.
>
> If you have custom implementations, add:
>
> ```
> public void Dispose() { /* cleanup */ }
> ```

5.2.2 Recommended Changes

```
// Remove manual locking (now thread-safe)
// BEFORE:
lock (_lock) { var svc = container.Resolve<IService>(); }

// AFTER:
var svc = container.Resolve<IService>();  // Thread-safe
```

5.3 Migration Checklist

☐ Update package reference to V2.1.0
☐ Add `Dispose()` to custom `IHandlingPromiseSource` implementations (if any)
☐ Add `PromiseSourcePools.PreWarm()` to startup (optional)
☐ Remove manual locking around dependency resolution (optional)
☐ Remove custom pooling implementations (optional)
☐ Run tests to verify functionality
☐ Run memory profiler to verify allocation reduction

6 Potential Issues & Considerations

6.1 Pool Memory Overhead

Pool	Max Size	Notes
TryHandlePromiseSource	64 × 128 B	8 KB max
TryHandleAsyncPromiseSource	64 × 128 B	8 KB max
HandlePromiseSource	64 × 128 B	8 KB max
HandleAsyncPromiseSource	64 × 128 B	8 KB max
Total	**32 KB**	Maximum pool memory

6.2 Struct Size Increase

`HandlingResultPromise` and related structs are slightly larger due to direct result fields:

- `HandlingResultPromise`: +16 bytes (result struct + bool)
- `HandlingResultAwaitable`: +16 bytes (result struct + bool)

Impact: Minimal — these are stack-allocated and short-lived.

6.3 Exception Behavior

> **Unchanged Behavior**
>
> Exception handling behavior is unchanged:
>
> - `Handle()` throws raw exceptions for sync failures
> - `HandleAsync()` wraps exceptions in `AggregateException`
>
> This matches the V2.0.0 and V1.0.0 behavior.

7 Glossary

Direct Result Path

The V2.1.0 optimization where `HandlingResultPromise` stores the result inline instead of in a promise source, eliminating allocation.

Pool Pre-Warming

Creating pool instances at startup to avoid first-use allocation spikes during normal operation.

Promise Source

The internal mechanism that tracks message handling completion and invokes callbacks.

Zero-Allocation

A code path that performs no heap allocations, avoiding garbage collection pressure entirely.

ConcurrentBag<T>

A thread-safe, unordered collection optimized for scenarios where the same thread produces and consumes items.

GC Pressure

The rate at which objects are allocated. High GC pressure leads to frequent garbage collections.

Sync Handler

A message handler that returns results synchronously (e.g., `ISyncMessageHandler<T>`).

Async Handler

A message handler that returns results asynchronously via `Task` (e.g., `ITaskMessageHandler<T>`).

8 Appendix

8.1 V2.1.0 Files Changed

File	Type	Lines Δ
`HandlingResultPromise.cs`	Modified	+60
`HandlingResultAwaitable.cs`	Modified	+55
`HandlingCompletionPromise.cs`	Modified	+40
`HandlingCompletionAwaitable.cs`	Modified	+40
`ISyncMessageHandler.cs`	Modified	−5
`ISyncContextHandler.cs`	Modified	−5
`IMessageHandler.cs`	Modified	+20
`IContextHandler.cs`	Modified	+20
`TryHandlePromiseSource.cs`	Modified	+20
`TryHandleAsyncPromiseSource.cs`	Modified	+20
`HandlePromiseSource.cs`	Modified	+20
`HandleAsyncPromiseSource.cs`	Modified	+20
`PromiseSourcePools.cs`	**New**	85

8.2 Test Results

```
Dependencies: 84 tests passed
Messages:     79 tests passed
──────────────────────────────
Total:       163 tests passed, 0 failed
```

✓ Zero-allocation sync path verified
✓ All pooling tests passing
✓ Backward compatibility confirmed
✓ Ready for production

8.3 Version History

Version	Date	Summary
2.1.0	May 2026	Zero-allocation sync path, full pooling, pre-warming
2.0.0	May 2026	GC reduction, thread safety, algorithmic improvements
1.0.0	—	Original release

Chopsticks Framework

High-Performance Dependency Injection
& Message Handling for .NET

Zero-Allocation Sync Path

Version 2.1.0
May 2026

For questions or support:
engineering@company.com